Team Deliverable 5 Report

Brian Tramuel, Will Armentrout, Sam Suther, Isaac Vunzi, and Sana Ullah
Library Management System

A. Phase 3 Requirements

The scope of Phase 3 is unchanged from the plan established in Team Deliverable 2. Phase 3 builds directly on the authentication, catalog, and borrow/return workflows delivered in Phases 1 and 2. This phase focuses on admin tools, overdue management, reservation functionality, and UI polish. The requirements below are the same Phase 3 requirements specified in Deliverable 2, expanded with detailed acceptance criteria and implementation notes.

Phase Overview

Phase	Focus	Included Requirements
Phase 1	MVP Auth and Catalog	FR-1 to FR-8, NFR-1 to NFR-4, NFR-9, NFR-10
Phase 2	Core Security, Borrow and Return	FR-9 to FR-16, NFR-5 to NFR-7
Phase 3	Admin Tools & Polish	FR-17 to FR-23, NFR-8, NFR-11, NFR-12

Functional Requirements

ID	Name	Detailed Description	
FR-17	Admin Book Management	Admin users can add new books, edit book details (title, author, genre, ISBN, total copies), and soft-delete books from the catalog. All changes are immediately reflected in the catalog view. Soft-deleted books are hidden from users but retained in the database for audit purposes.	
FR-18	Admin User Management	Admin users can view a complete list of registered accounts, promote users to the admin role, or deactivate accounts. Deactivated accounts cannot log in, but their borrowing history is preserved.	
FR-19	Overdue Detection and Notifications	The system identifies loans where the due date has passed, and the status is still 'active', automatically marking them 'overdue'. Users see an overdue badge on their loan list. A server-side scheduled job runs daily to update overdue statuses.	
FR-20	Book Reservation / Waitlist	When a book has zero available copies, authenticated users can join a waitlist. When a copy is returned, the first user on the waitlist is notified and given 48 hours to borrow before the slot is released to the next user.	

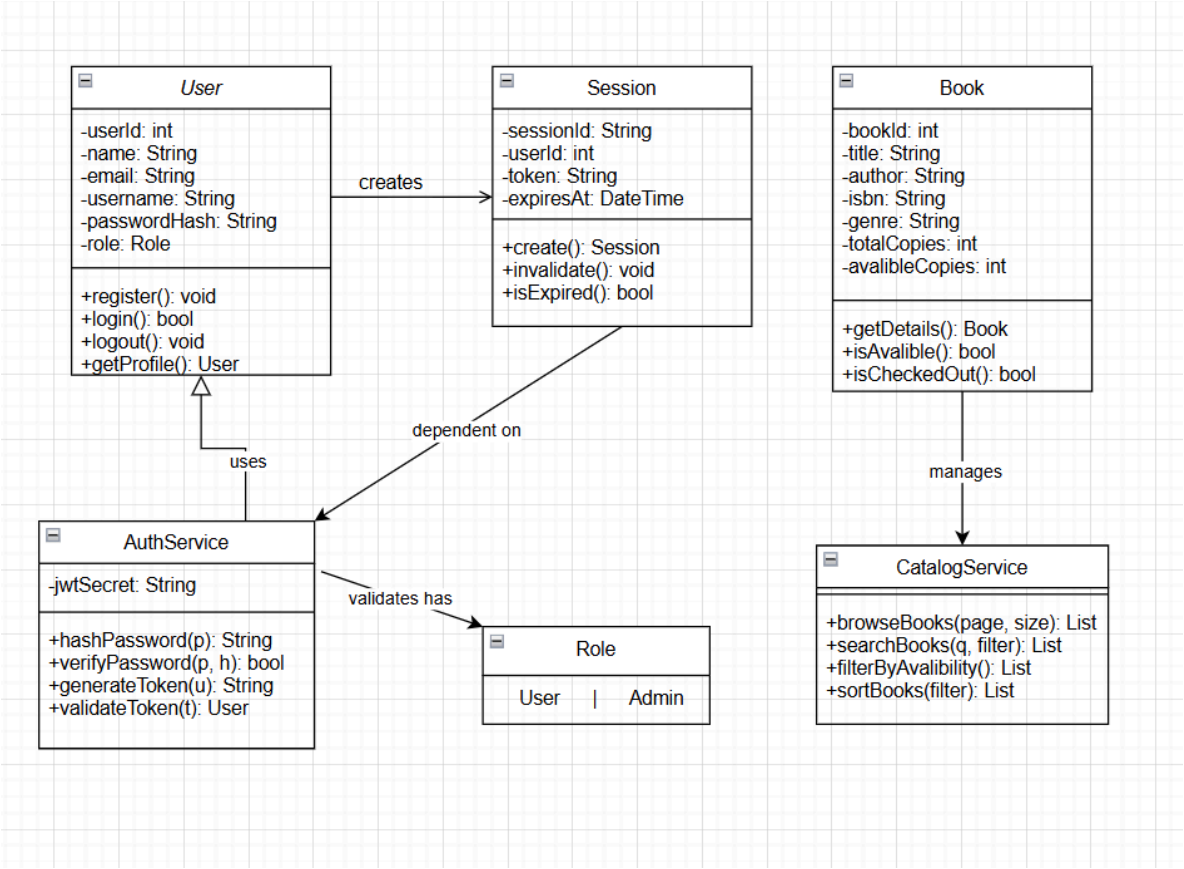
FR-21	Borrow History and Reporting	Admin users can access GET /api/admin/reports to view system-wide borrowing statistics, including total loans, most borrowed books, overdue rates, and active users. Users can view their complete personal history at GET /api/borrow/my, including returned and overdue records.	
FR-22	Book Cover Images	Books in the catalog display a cover image. Admins can upload a cover image when creating or editing a book. Images are stored in a designated directory and referenced by path in the database. A placeholder image is shown when no cover is available.	
FR-23	Search and Filter Enhancements	The catalog search supports filtering by genre dropdown, availability toggle, and sorting by title, author, or most recently added. All filters are composable, and results update without a full page reload.	

Non-Functional Requirements

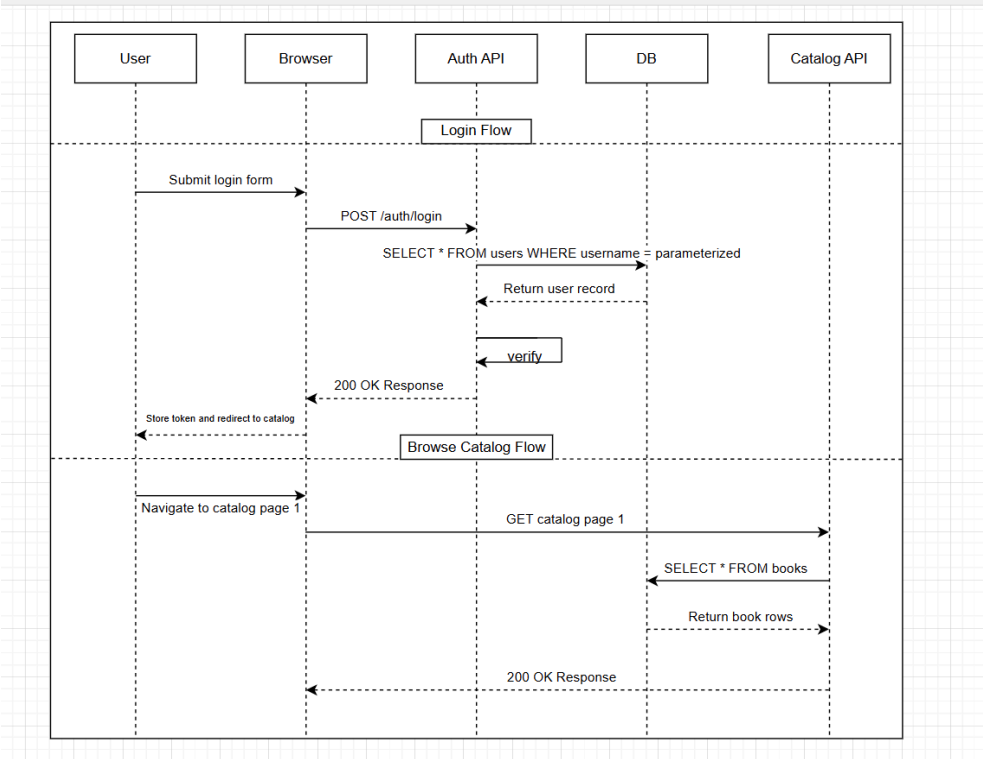
ID	Category	Requirement and Detail
NFR-8	Usability	The admin dashboard provides a clearly labeled, single-page interface for all management tasks. All user-facing error messages are written in plain language (no raw SQL or stack traces). Page load times remain under 2 seconds for admin views.
NFR-11	Maintainability	The code base is organized into modules, routes, controllers, middleware, database with consistent naming conventions and a JSDoc style header with comments on each file describing its responsibilities. The shared logic with the database connection pooling and JWT verification is centralized to avoid duplication, and configuration values are read from environment variables rather than being hardcoded.
NFR-12	Security	Admin routes are protected by a role-check middleware that verifies the JWT role claim equals 'admin' before processing any request. File upload endpoints validate MIME type and limit file size to 5 MB to prevent malicious uploads.

B. UML Design For Phase 1

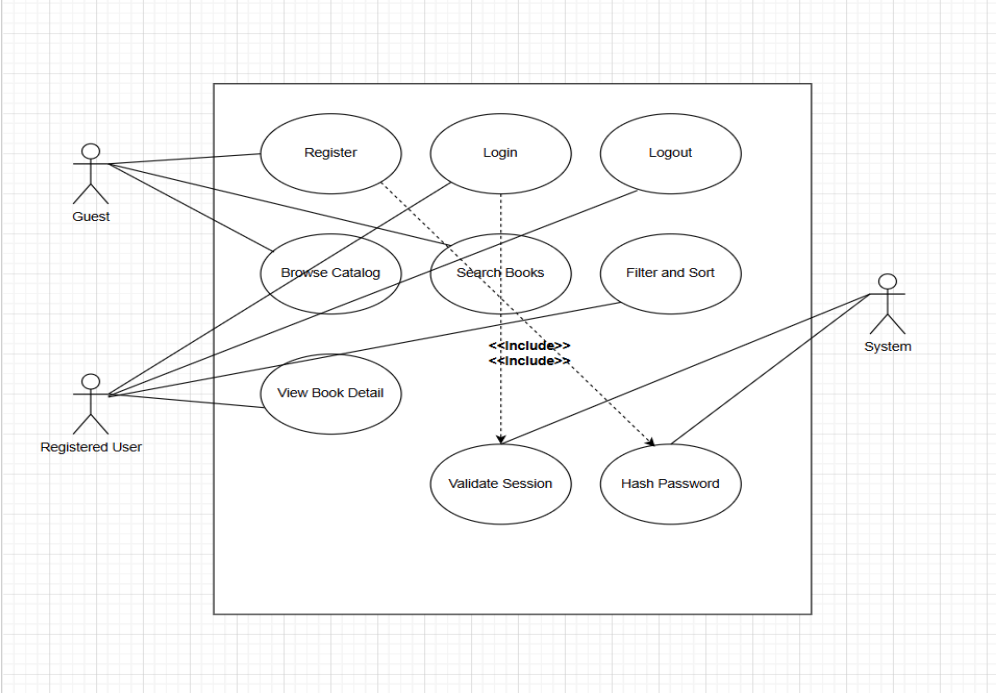
Class Diagram



Sequence Diagram



Use-Case Diagram



C. User Manual

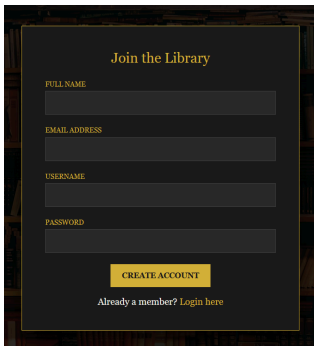
This manual explains how to access and use the Library Management System as it currently exists after Phase 3. The application is a web-based system with a Node.js/Express backend, a SQL database, and an HTML/CSS/JavaScript frontend styled with a dark theme.

Accessing the Application

Once the server is running, open your browser and go to *http://localhost:3000*. You will see the Library home page with a centered layout, a Dark Academia-themed design, and navigation links for Login, Register, and Search in the header.

Registering a New Account

1. Click **Register** in the navigation bar. The registration page will load.
2. Fill in your **Full Name, Email Address, Username, and Password**.
3. Your password must be at least **8** characters and include one uppercase letter, one lowercase letter, one digit, and one special character.
4. Click **Create Account**. On success, you are redirected to the login page. If the username or email is already taken, an error message will appear below the field.



Logging In and Out

Click Login in the navigation bar. Enter your username and password, then click Sign In. After a successful login, your session is stored, and you are redirected to the catalog. To log out, click the Logout link in the navigation bar; your session will be cleared immediately.

Searching and Browsing the Catalog

The Search page is accessible from the navigation bar at any time, whether or not you are logged in.

- **Search bar**: Type a book title or author name and press Enter or click Search. Results update on the page without a full reload.
- **Browse all**: Leave the search bar empty and submit to see all available books.
- **Book cards**: Each result card shows the title, author, genre, and availability status.
- **Book Detai**l: Click a book title or card to open its full detail page, which lists ISBN, genre, total copies, and available copies.

Borrowing and Returning Books

Authenticated users can borrow up to three books at a time. On a book's detail page, click Borrow to check out an available copy. The system records a due date 14 days from the borrow date. To return a book, navigate to My Loans and click Return next to the active loan. The availability count updates immediately.

Viewing Your Loan History

Click My Loans in the navigation bar to see all of your active, returned, and overdue loans. Overdue loans display a red Overdue badge. You can view due dates and borrowing dates for every transaction in your history.

Book Reservations / Waitlist

If a book has no available copies, authenticated users will see a Join Waitlist button on the detail page. When a copy is returned, you will be notified and given 48 hours to borrow the book before the slot passes to the next user on the waitlist.

Admin Dashboard

Users with the Admin role have access to the Admin Dashboard, reachable via the Admin link in the navigation bar. The dashboard provides a single-page interface for all administrative tasks:

- **Book Management:** Add new books, edit existing book details, or soft-delete a book from the catalog. Soft-deleted books are hidden from users but retained in the database.
- **User Management:** View all registered accounts, promote a user to the Admin role, or deactivate an account. Deactivated accounts cannot log in, but their borrowing history is preserved.
- **Reports:** View system-wide statistics, including total loans, most borrowed books, overdue rates, and active user counts via GET /api/admin/reports.

D. Compile/Run Instructions

The project uses Node.js and Express for the backend, PostgreSQL for the database, and plain HTML/CSS/JavaScript for the frontend. All source files are on the main branch of the GitHub repository.

Must Have:

- **Node.js**
- **Git**
- **PostgreSQL**

Step 1 - Clone Git

git clone git clone https://github.com/ssuther6-ops/ITIS3300-Panthers.git cd ITIS3300-Panthers

Step 2 - Install Dependencies

The project dependencies are declared in package.json.
Install them with: **npm install**

Step 3 - Create and Configure the Database

Open a psql shell and create the database:
CREATE DATABASE olms;

Then run the schema script to create the tables:
psql -d olms -f src/database/schema.sql

Then load the seed data (sample books and admin user):
psql -d olms -f src/database/seed.sql

Step 4 - Set Environment Variables

Create a .env file in the project root and set the following:

Variable	Example Value
DB_Host	localhost
DB_Port	5432
DB_Name	olms
DB_User	postgres
DB_Password	Passw0rd
JWT_Secret	panthers3300
PORT	3000

The .env file isn't committed to the repo for security reasons; you have to create it yourself using the values above

Step 5 - Start the Server

node src/backend/server.js

You should see: OLMS server port 3000. Open *http://localhost:3000* in your browser to use the application.

API Endpoints

EndPoint	Method
/api/auth/register	POST
/api/auth/login	POST
/api/books	GET
/api/books?search=query	GET
/api/borrow	POST
/api/borrow/return	POST

/api/borrow/my	GET
/api/users	GET
/api/users/history	GET

E. Test Cases

#	Method	Input	Expected Output	Pass or Fail
1	POST /api/auth/register **(valid new user)**	{name:"Jane", email:"j@x.com", username:"jane1", password:"Pass@123" }	201 Created; user object returned; no password hash in response	Pass
2	POST /api/auth/register **(duplicate username)**	{ same username as test 1 }	400 Bad Request; 'Username or email already taken.'	Pass
3	POST /api/auth/register **(weak password)**	{ password: "abc" }	400 Bad Request; password complexity error message	Pass
4	POST /api/auth/login **(valid credentials)**	{ username:"jane1", password:"Pass@123" }	200 OK; JWT token returned in response body	Pass
5	POST /api/auth/login **(wrong password)**	{ username:"jane1", password:"wrong" }	401 Unauthorized; 'Invalid username or password.'	Pass
6	GET /api/books **(no auth, no filter)**	No body, no token	200 OK; JSON array of all active books sorted A-Z	Pass
7	GET /api/books?search =gatsby **(search filter)**	Query param search=gatsby	200 OK; only books with 'gatsby' in title or author returned	Pass
8	GET /api/books?	Query param available=true	200 OK; only books with	Pass

	available=true **(availability filter)**		available_copies > 0 returned	
9	POST /api/borrow **(borrow an available book)**	Valid JWT; { book_id: 1 }	201 Created; transaction row with due_date 14 days out	Pass
10	POST /api/borrow **(Borrow the same book twice)**	Same JWT and book_id as test 9	400 Bad Request; 'You already have this book borrowed.'	Pass
11	POST /api/borrow **(book unavailable)**	Valid JWT; book_id with available_copies=0	400 Bad Request; 'No copies available.'	Pass
12	POST /api/borrow **(exceed 3-book limit)**	Valid JWT; 4th borrow attempt	400 Bad Request; 'Borrow limit of 3 reached.'	Pass
13	POST /api/borrow/return **(return active loan)**	Valid JWT; { transaction_id: 1}	200 OK; 'Book returned successfully'; available_copies incremented	Pass
14	POST /api/borrow/return **(return someone else's loan)**	Different user JWT; same transaction_id	404 Not Found; 'Active transaction not found.'	Pass
15	GET /api/borrow/my **(view own loans)**	Valid JWT; no body	200 OK; an array of the user's borrow records with title, due_date, status	Pass
16	GET /api/users **(admin route, non-admin token)**	User-role JWT	403 Forbidden; 'Admin access only.'	Pass
17	GET /api/users	Admin-role JWT	200 OK; array of all user	Pass

	(admin route, admin token)		accounts; no password_hash fields	
18	GET /api/auth/profile **(valid token)**	Valid JWT	200 OK; authenticated user's profile without password hash	Pass
19	GET /api/auth/profile **(no token)**	No Authorization header	401 Unauthorized; 'Access denied. No token provided.'	Pass

F. Implemented Features and Limitations

Successfully Implemented Features

- User registration and login with JWT-based session management (Phase 1).
- Book catalog browsing, searching by title/author, and filtering by availability (Phase 1).
- Full borrow and return workflow with atomic PostgreSQL transactions (Phase 2).
- Borrowing limit enforcement (maximum 3 active loans per user) is enforced server-side (Phase 2).
- Duplicate-loan prevention — users cannot borrow the same book twice concurrently (Phase 2).
- Loan history view with due dates and statuses at GET /api/borrow/my (Phase 2).
- Role-based access control: admin-only routes return 403 for non-admin users (Phase 2).
- Code inspection feedback addressed: named color constants, corrected path comments, JSDoc on MAX_BORROW_LIMIT, and explicit null checks in auth middleware (Phase 2).
- Admin dashboard with single-page interface for book CRUD, user management, and system reports (Phase 3).
- Overdue detection scheduled job — runs daily to mark loans overdue; overdue badge displayed on loan list (Phase 3).
- Book reservation/waitlist system with 48-hour claim window and notification on copy return (Phase 3).
- Book cover image upload and display with placeholder fallback (Phase 3).
- Advanced search filters: genre dropdown, availability toggle, and multiple sort options; all filters are composable, and results update without full page reload (Phase 3).
- Frontend borrow/return buttons connected to live API endpoints with real-time availability badge updates (Phase 3).

Known Limitations and Issues

- **Automated testing:** All test cases are run manually with Postman. There is no automated regression suite; a code change can break an existing test without immediate detection.
- **Feature branching:** Most commits across all phases went directly to main. This increases the risk of conflicts and makes code review harder.
- **Email notifications:** Waitlist and overdue notification emails are simulated; a real email delivery service has not been integrated.
- **Performance under load:** Load testing beyond manual curl requests has not been performed. NFR-5 and NFR-6 compliance is assumed but not verified with formal tooling.

G. Reflection

What Has Been Accomplished

Phase 3 is complete. All planned features have been implemented: the admin dashboard (FR-17, FR-18, FR-21), the overdue detection scheduled job (FR-19), the book reservation and waitlist system (FR-20), book cover image upload and display (FR-22), and advanced search filters (FR-23). The frontend borrow/return UI stubs from Phase 2 have been connected to the live API endpoints, and the availability badge now updates in real time. Admin routes are protected by role-check middleware verifying the JWT role claim, and file upload endpoints validate MIME type and enforce a 5 MB limit per NFR-12.

What Went Well

- **Admin dashboard:** The single-page admin interface came together cleanly, with all management tasks accessible without navigation away from the dashboard.
- **Scheduled job:** The daily overdue detection job was implemented using a lightweight scheduler and integrates without adding latency to user-facing endpoints, resolving the on-read computation issue noted in Phase 2.
- **Waitlist system:** The 48-hour claim window logic was straightforward to implement, given the atomic transaction patterns established in Phase 2.
- **Frontend integration:** Connecting the borrow/return buttons to the live API, the top Phase 3 priority was completed first, giving the team a working end-to-end flow early in the phase.
- **Code quality:** The pull-request workflow introduced in Phase 3 reduced direct-to-main commits and allowed meaningful peer review before merging.

Areas for Improvement

- **Automated testing:** All test cases were still run manually. A Jest or Mocha suite should be introduced so tests run automatically on every commit.

- **Email delivery:** Notification emails are not sent through a real delivery service. A future iteration should integrate a transactional email provider.
- **Load testing:** Formal load testing has not been performed. Verifying NFR-5 and NFR-6 compliance with a tool such as k6 or Apache JMeter should be a post-Phase-3 priority.

H. Contribution Table

Member Name	Contribution Description	Overall Contribution %	Note
Will	Team Deliverable 5		
Brian			
Isaac			
Sam			
Sana			